Parley — 设计文档

Oct 30, 2026 · @EvATive7

命名

名称：**Parley**。英文原意是双方坐下来谈判、交谈，正好对应两个说不同语言的人对话。只有两个音节，好读好记。

- 包名：`io.github.evative7.parley`
- 备选：**Utter**（说出口即翻译）、**Tandem**（两人一前一后接力对话）
- 开源发布前，在 GitHub 上搜一下有没有同名的安卓仓库

产品概述

Parley 是一个开源的安卓语音翻译 App：按住按钮说话，语音经云端 ASR 转成文字，再交给 LLM 翻译，译文逐字显示。卖点是交互顺手、动画好看、翻译自然。

发布方式

- 个人自用，代码在 GitHub 开源，APK 通过 GitHub Releases 发布；暂不上架应用商店。
- 没有后端服务：ASR 和 LLM 的 Key 由用户在 App 里自己填写。
- 最低支持 Android 8.0（API 26）。

核心场景

1. 单按钮模式：顶部选双方的两种语言，一部手机轮流按住说话，识别出哪种就译成另一种（语音播报等 TTS 接入后再加）。
2. 对话模式：屏幕上下两个按钮，双方各按自己的一个，上半屏旋转 180° 给对面的人看。

语言：不单独限定。语言栏选双方各一种语言，可以是 LLM 能输出的任意语言。说话的语言不在界面上选，由 ASR 自动识别：识别出语言 2 就译成语言 1，其余情况（包括两种都不是）都译成语言 2。

V1 范围

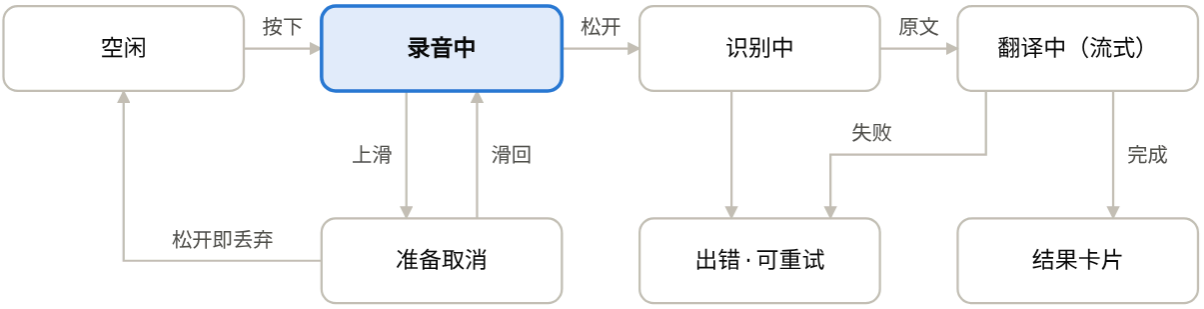
做	不做
长按录音、上滑取消、形变动画	离线 ASR / 离线 LLM
4 家云端 ASR + OpenAI 兼容接口流式翻译	实时同传（边说边译）
单人模式 + 对话模式	拍照翻译、文档翻译
本地历史、复制、删除	译文 TTS 播报（V1 只留接口）

做	不做
	账号体系与云同步
	iOS 版、应用商店上架

交互与动画设计

整个 App 围绕屏幕底部一个 96 dp 的可变形麦克风按钮（外观见“视觉设计”一节）：按住录音，上滑取消，松开翻译。交互参考微信语音，用户无需学习。

按住即录，上滑取消，松开即翻译



长按说话状态机 · 7 个状态

上滑超过 80 dp 进入“准备取消”，滑回恢复录音。在结果卡片或出错状态下再次按下，直接进入录音中。

各状态的视觉与反馈

状态	按钮	屏幕	触感
空闲	九瓣饼干形，每 12 s 慢转一圈	最近的结果卡片	—
录音中	形变为放射形并放大到 1.15 倍；外圈波浪进度环显示已录时长，波幅随音量变化	新卡片入场；流式 ASR 时实时显示原文	按下时轻振
准备取消	变成红色圆形并放大到铺满进度环，图标换成删除	提示“松开取消”	进入时振一次
识别中	按钮本身变成 Loading Indicator（形状循环变化）	原文位置显示骨架屏（流式 ASR 时跳过）	—
翻译中	同上	原文定格；译文逐字出现	—

状态	按钮	屏幕	触感
结果卡片	恢复空闲形状	卡片底部出现图标按钮：复制、删除	—
出错	恢复空闲形状	卡片内显示原因 + 重试、删除	拒绝类振动

底部双语提示：按钮上方一直有提示，双方语言各一行，拿着手机的哪一方都看得懂。切换时统一上行渐隐。

状态	提示
空闲 / 翻译中 / 结果 / 出错	按住说话
录音中	松开发送 · 上滑取消
录音最后 10 s	还剩 N 秒
准备取消	松开取消（错误色）
识别中	识别中…
复制后 2 s（Android 12 及以下）	已复制

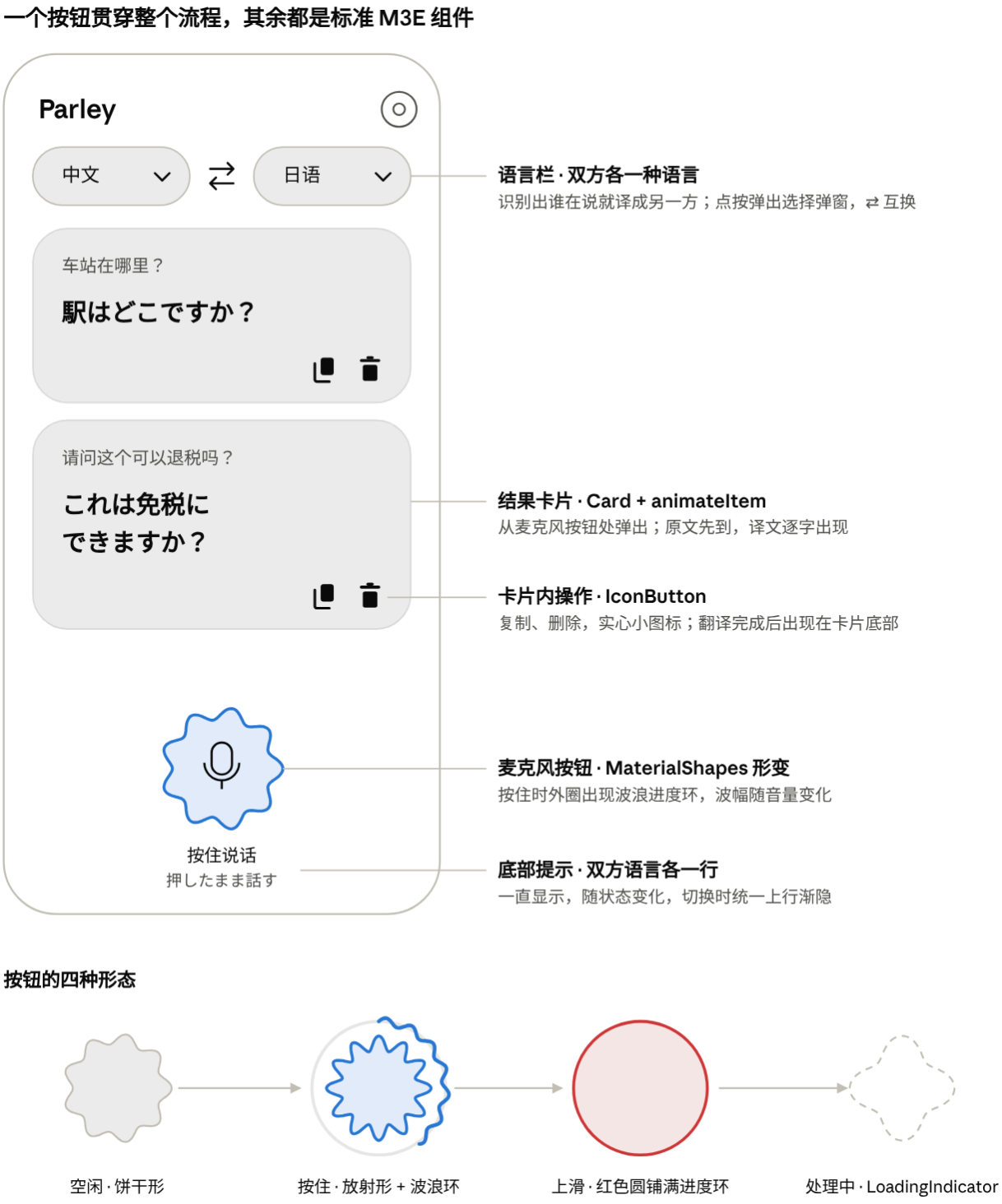
按住不到 0.5 s 被当作误触丢弃，提示回到“按住说话”并重播一次动画，不再单独弹提示。

实现要点

- 手势：`pointerInput` + `awaitEachGesture`。手指一接触就开始录音，不等长按判定；不到 0.5 s 就松开则丢弃。这样按下到开始录音 ≤ 100 ms，第一个字不会被吃掉。
- 音量：每 20 ms 一帧 PCM 算 RMS，转成 dB 后映射到 0–1，再平滑，用来驱动形变和波幅。
- 译文流式：每收到一个片段就追加，新字用 150 ms 淡入。
- 对话模式：上下两个按钮各带语言标签，上半屏内容 `rotate(180f)`。

视觉设计（Material 3 Expressive）

只用 M3 Expressive 的官方组件、形状、动效 token 和配色角色，不自造控件。巧思放在两处：让同一个元素贯穿整个流程，以及用规范组件承载实时数据。



主界面示意 · 按钮的 4 种形态

语言栏、卡片和卡片内的图标按钮都是规范组件；只有底部按钮会变形，一个元素就告诉用户现在处于哪一步。

主题

- `MaterialExpressiveTheme` + `MotionScheme.expressive()`：位移和形变用 spatial spring，颜色和透明度用 effects spring。

- 配色：Android 12+ 用动态取色；Android 8–11 用一个种子色生成的固定配色。深浅色都支持。
- 颜色角色：空闲按钮 `primaryContainer`，录音时 `primary`，准备取消 `errorContainer`；对话模式下双方分别用 `primaryContainer` 和 `tertiaryContainer`，一眼分清谁在说。
- 字体：译文用 `headlineSmallEmphasized`，原文用 `bodyLarge` + `onSurfaceVariant`，译文始终是视觉重点。
- 全面屏（edge-to-edge），Android 13+ 支持预测性返回。

巧思

设计	用到的规范元素	效果
一个按钮讲完整个流程	`MaterialShapes` + `Morph`：饼干形 → 放射形 → 红色圆形 → `LoadingIndicator`	视线不离开拇指就知道进度
波浪进度环兼作音量表	`CircularWavyProgressIndicator`：进度 = 已录 / 60 s，波幅 = 音量	一个规范组件同时表达时长和音量
原文先到	流式 ASR 边录边出字	松开时卡片里已有原文，只等翻译
语言栏	两个 Tonal Button + 互换按钮 + `BasicAlertDialog`：[语言 1 ▾] ⇄ [语言 2 ▾]	弹窗里每种语言用自己的文字写（如“日本語”），对方也能找到自己的；选到另一栏的语言就互换
卡片从按钮弹出	`LazyColumn` + `animateItem()`，卡片用 `extraLarge` 圆角	新结果像是从按钮里“吐”出来的
结果操作	卡片底部一行 `IconButton`：复制、删除（实心图标，20 dp）	翻译完成才出现，只用图标、不带文字
双语底部提示	`AnimatedContent` 上行渐隐；用 `createConfigurationContext` 按语言取字符串，15 种语言各有一份	文字只往一个方向走，倒计时像翻牌一样逐秒升起
引导页	大号 `MaterialShapes` 插图 + 一个 Filled Button	授权成功后插图形变成对勾，再进入主页
触感	`HapticFeedbackConstants`：按下 `VIRTUAL_KEY`，成功 `CONFIRM`，失败 `REJECT`	`CONFIRM` / `REJECT` 需 Android 11+，以下降级为普通短振

无障碍与降级

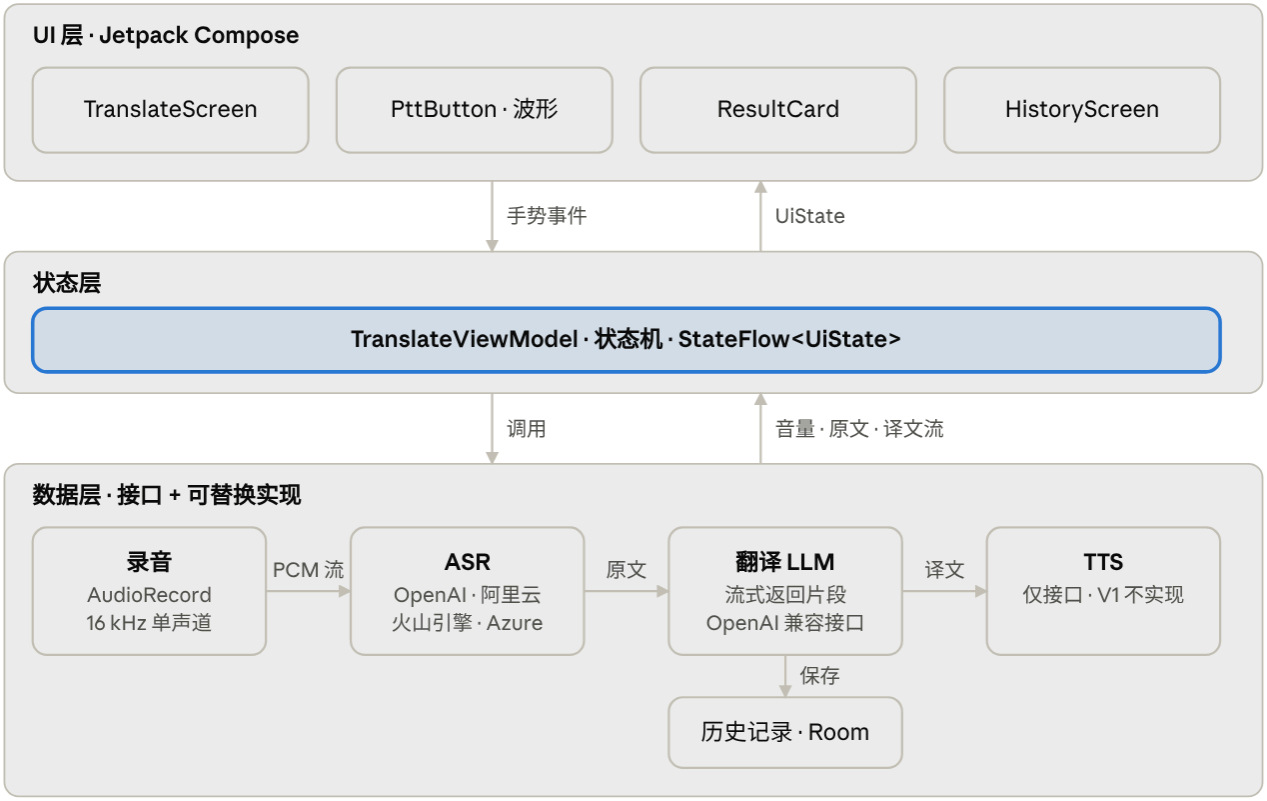
- TalkBack 开启时，长按改为“点一下开始、再点一下结束”，并朗读状态变化。

- 系统“移除动画”开启时改用 `MotionScheme.standard()`，关掉慢转和波浪。
- 所有可点击区域 ≥ 48 dp；文字颜色只用 `on*` 配色角色，保证对比度。
- 依赖 `androidx.compose.material3` 1.4+，表达性 API 大多标注 `@ExperimentalMaterial3ExpressiveApi`，版本要锁定。

技术架构

Kotlin + Jetpack Compose，单 Activity、MVVM、单向数据流。UI 只根据 `UiState` 绘制；ASR、翻译、TTS 都是接口（TTS V1 无实现），各家服务可以在设置里随时切换。

UI 只看状态，ASR 与翻译服务可随时切换



分层架构 · 3 层

录音时 PCM 分两路：一路实时算音量驱动动画；另一路在流式服务下边录边上传，在整段服务下先缓存，松开后一次性上传。

技术栈

用途	选型
UI 与动画	Jetpack Compose、Material 3 Expressive（material3 1.4+）、androidx.graphics.shapes

用途	选型
异步	Kotlin Coroutines + Flow
依赖注入	Hilt
网络	OkHttp（HTTP、SSE、WebSocket）、kotlinx.serialization
本地存储	Room（历史）、DataStore（设置）
密钥存储	Android Keystore 加密后存入 DataStore

工程结构

```
app/
├─ ui/          TranslateScreen, PttButton, ResultCard, History, Onboarding,
Settings
├─ state/       TranslateViewModel, UiState, PttStateMachine
├─ audio/       AudioRecorder, VolumeMeter (RMS)
├─ asr/         SpeechRecognizer, OpenAiRecognizer, DashScopeRecognizer,
│               VolcengineRecognizer, AzureRecognizer
├─ translate/ Translator, OpenAiCompatibleTranslator (SSE)
├─ tts/         TextToSpeaker（仅接口，V1 无实现）
└─ data/        HistoryDao, SettingsRepository, KeyVault
```

ASR 与 LLM 方案选型

V1 用云端 ASR + 云端 LLM，Key 由用户自己填写。ASR 沿用 Voxaia 的四家服务，在设置里切换；服务支持流式就优先用流式。

ASR

服务	模型	方式	说明
OpenAI	`gpt-realtime-whisper`	流式（Realtime WebSocket）	边说边出原文
OpenAI	`gpt-4o-transcribe` / `gpt-4o-mini-transcribe`	整段上传	多语种，接入最简单
阿里云 DashScope	`fun-asr-realtime`	流式 WebSocket	支持热词
火山引擎	豆包 Seed ASR 2.0：流式 SAUC；整段 AUC 标准版 / 极速版	流式或整段	新控制台的 `X-Api-Key` 目前只适用于录音文件识别

服务	模型	方式	说明
Azure	MAI-Transcribe-1、Fast Transcription	整段上传	Voxaia 里有文档，还没有实现

每家支持的语言列表写成静态配置，语言栏直接读取。

翻译与 TTS

- 翻译：任意 OpenAI 兼容的 Chat Completions 接口（Base URL + Key + 模型名），OpenAI、DeepSeek、Groq、通义千问都能接。
- TTS：V1 不接入，只保留统一接口 `TextToSpeaker`；以后可接云端 TTS 或系统 `TextToSpeech`。

接口抽象

- `SpeechRecognizer.open(lang: String?): RecognitionSession`
- `RecognitionSession.feed(pcm: ShortArray)`、`partials: Flow<String>`、`finish(): Transcript`、`cancel()`；整段服务在 `finish()` 里才上传
- `Translator.translate(text: String, from: Lang?, to: Lang): Flow<String>`（流式返回片段）
- `TextToSpeaker.speak(text: String, lang: Lang): Flow<PlaybackState>`、`stop()`；V1 只定义接口、不提供实现，UI 不显示播放入口

翻译 Prompt

```
You are a professional interpreter. Translate the user's speech from {from}
to {to}.
- Output only the translation, no explanations or quotes.
- Keep the tone and register of the speaker (casual stays casual).
- The text comes from speech recognition and may contain errors; infer the
intended meaning.
- Keep names, numbers and units accurate.
```

温度设为 0.2–0.3。源语言固定为自动检测，由 ASR 返回的语种决定。对话模式下，每次请求附带最近 3 轮原文作为上下文，提升指代和术语一致性。

数据、隐私与权限

只申请麦克风和网络两个权限；录音发送完就删除，只保存文字。

- **权限**：每次打开首屏时静默检查 `RECORD_AUDIO`，不弹窗。未授权就进入引导页：说明用途，配一个“授予麦克风权限”按钮；被永久拒绝时按钮改为“前往系统设置”，从设置返回后自动重查。授权后进入主页。

- **首次启动引导**：麦克风权限 → 选 ASR 服务并填 Key → 填翻译接口（Base URL、Key、模型）。每一步都有“测试连接”。
- **历史记录**：Room 本地库，字段为 id、时间、源语言、目标语言、原文、译文；支持一键清空。
- **音频**：只存在缓存目录，识别完成或取消后立即删除。
- **API Key**：用户在设置里填写，用 Android Keystore 加密后存入 DataStore。仓库和 APK 里不带任何 Key，也没有自建后端。
- **隐私说明**：README 和设置页写明：语音和文字会直接发给用户所选的云服务。

性能指标与错误处理

核心指标：松开手指到译文第一个字出现 ≤ 1.5 秒（一句 5 秒内的话，4G/Wi-Fi）。以下为设计目标，需实测校准。

指标	目标
按下到开始录音	≤ 100 ms
动画帧率	60 fps
松开到原文出现	≤ 1 s
松开到译文首字	≤ 1.5 s
单次录音上限	60 s，剩 10 s 时倒计时提示

情况	处理
按住 < 0.5 s	视为误触，提示“按住说话”，不发请求
全程静音（VAD 未检测到人声）	提示“没听清，再试一次”
无网络	按钮上方显示离线条；录音仍可用，结果卡片显示“重试”
ASR / LLM 超时（> 10 s）	自动重试 1 次，再失败则卡片显示错误与重试按钮
译文生成中又按下新一句	取消上一次流式请求，保留已生成部分
来电 / 切到后台	立即结束录音并丢弃，不发送

里程碑与开放问题

分三个阶段推进，每个阶段结束都发一个 GitHub Release。

1. **M1 交互原型**（已完成，2026-10-30）：M3E 主题、形变按钮、波浪进度环、上滑取消、权限引导页、双语语言栏与底部双语提示，结果用假数据。
2. **M2 跑通链路**：先接一家流式 ASR 和 OpenAI 兼容翻译，卡片实时出原文、逐字出译文。
3. **M3 完整版**：其余 ASR 服务、设置与 Key 管理、历史记录、对话模式、错误处理、无障碍。

已定：个人自用、用户自填 Key；GitHub 开源，暂不上架；最低 Android 8.0（API 26）；语言跟随 ASR；不做离线版；卡片去掉收藏和重说，只留复制和删除。

待定

- [ ] M2 先接哪家流式 ASR？（`gpt-realtime-whisper` 或 `fun-asr-realtime`）
- [ ] 翻译默认推荐哪个模型？
- [ ] 开源许可证：MIT、Apache-2.0 还是 GPL-3.0？